

Complete Analysis of GenAI Code

Name of the Code Files

main_v2.cpp (Gemini-generated)

ceo_email_priority.cpp(Claude-generated)

main.cpp (corrected/commented version, based on the Claude-generated code)

How They Were Accessed

Claude and Gemini, both accessed via their web browser interfaces (claude.ai and gemini.google.com).

Prompt

Write a C++ program that prioritizes emails for a busy company CEO.

Requirements:

- Implement a priority queue using a MaxHeap.
- The MaxHeap must use a list-based (array) implementation.
- Use the MaxHeap to handle all email prioritizing for the CEO.
- All heap functions must be written from scratch — do not use any pre-existing heap or priority-queue library or module.

Input file format:

The program reads a test file containing the following commands, one per line.

EMAIL <sender category>,<subject line>,<date>

The emails in the CEO's inbox should be placed in the queue based on their sender category and date. The sender categories and priority to be read are as follows: Boss reads first. Subordinate reads next. Peer reads next. ImportantPerson reads next. OtherPerson reads last. If there is more than one email from a sender, then the newest email (not the oldest) should be read first. EMAIL is followed by a space. The rest of the fields are comma-delimited. Assume sender category is one of the five strings listed above. Assume subject line is a string which may contain spaces but not commas. Assume date is in the format MM-DD-YYYY.

NEXT

Shows the next email for the CEO to read. Display the information on the terminal in the following format: Sender, Subject, Date.

READ

The CEO has read the email and dealt with it. No output is produced.

COUNT

Display the current count of unread emails.

The grader will test the code with a different test file. The test file will begin with the emails in the CEO's inbox, followed by the commands the CEO will use to read her inbox. The order and number of commands may vary. The program must be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (the output should be the same for both NEXTs), and two READs in a row without an intervening NEXT (the highest priority email is deleted from the queue without displaying it).

Code Generated: Claude

```
// CEO Email Prioritizer
// Uses a hand-built, array-based MaxHeap (no pre-existing heap
modules).
// Build:  g++ -std=c++11 -o ceo_email ceo_email_priority.cpp
// Run:    ./ceo_email input.txt          (or pipe a file to stdin)

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

//
-----
// Email: one message plus everything needed to rank it.
//
-----

class Email {
public:
    Email() : rank(0), year(0), month(0), day(0), seq(0) {}

    Email(const std::string& senderCategory, const std::string&
subjectLine,
          const std::string& dateText, long arrivalOrder)
        : sender(senderCategory), subject(subjectLine),
date(dateText),
        rank(rankFor(senderCategory)), year(0), month(0), day(0),
seq(arrivalOrder) {
        parseDate(dateText);
    }
}
```

```

    // True if this email should be read BEFORE the other one.
    bool hasHigherPriorityThan(const Email& other) const {
        if (rank != other.rank) return rank > other.rank;           //
sender category
        if (year != other.year) return year > other.year;           //
newest date first
        if (month != other.month) return month > other.month;
        if (day != other.day) return day > other.day;
        return seq < other.seq;                                       //
tie: arrived first
    }

    const std::string& getSender() const { return sender; }
    const std::string& getSubject() const { return subject; }
    const std::string& getDate() const { return date; }

private:
    std::string sender, subject, date;
    int rank;
    int year, month, day;
    long seq;

    // Strict category order: Boss > Subordinate > Peer >
ImportantPerson >
    // OtherPerson. Within a category, the newest date wins.
    static int rankFor(const std::string& category) {
        if (category == "Boss") return 5;
        if (category == "Subordinate") return 4;
        if (category == "Peer") return 3;
        if (category == "ImportantPerson") return 2;
        return 1;                                                     //
OtherPerson / unknown
    }

    void parseDate(const std::string& text) {                           //
MM-DD-YYYY
        int m = 0, d = 0, y = 0;
        char dash1 = 0, dash2 = 0;
        std::istringstream in(text);
        if (in >> m >> dash1 >> d >> dash2 >> y) {
            month = m; day = d; year = y;

```

```

        }
    }
};

//
-----
-----
// MaxHeap: array-based binary heap of Email objects, written from
scratch.
// Parent of i is (i-1)/2; children are 2i+1 and 2i+2.
//
-----
-----
class MaxHeap {
public:
    MaxHeap() : size(0), capacity(8), data(new Email[8]) {}
    ~MaxHeap() { delete[] data; }

    // Copying is not needed; disable it to avoid double-deletes.
    MaxHeap(const MaxHeap&) = delete;
    MaxHeap& operator=(const MaxHeap&) = delete;

    bool isEmpty() const { return size == 0; }
    int count() const { return size; }

    void insert(const Email& e) {
        if (size == capacity) grow();
        data[size] = e;
        siftUp(size);
        ++size;
    }

    // Highest-priority email without removing it. Caller must check
isEmpty().
    const Email& peek() const { return data[0]; }

    // Removes the highest-priority email. Caller must check
isEmpty().
    void removeMax() {
        --size;
        if (size > 0) {

```

```

        data[0] = data[size];
        siftDown(0);
    }
}

```

private:

```

    int size;
    int capacity;
    Email* data;

    void grow() {
        int newCap = capacity * 2;
        Email* bigger = new Email[newCap];
        for (int i = 0; i < size; ++i) bigger[i] = data[i];
        delete[] data;
        data = bigger;
        capacity = newCap;
    }

    void swapAt(int a, int b) {
        Email tmp = data[a];
        data[a] = data[b];
        data[b] = tmp;
    }

    void siftUp(int i) {
        while (i > 0) {
            int parent = (i - 1) / 2;
            if (data[i].hasHigherPriorityThan(data[parent])) {
                swapAt(i, parent);
                i = parent;
            } else {
                break;
            }
        }
    }

    void siftDown(int i) {
        while (true) {
            int left = 2 * i + 1, right = 2 * i + 2, best = i;

```

```

        if (left < size &&
data[left].hasHigherPriorityThan(data[best])) best = left;
        if (right < size &&
data[right].hasHigherPriorityThan(data[best])) best = right;
        if (best == i) break;
        swapAt(i, best);
        i = best;
    }
}
};

//
-----
-----
// CEOInbox: owns the MaxHeap and implements the EMAIL / NEXT / READ
/ COUNT
// commands.
//
-----
-----
class CEOInbox {
public:
    CEOInbox() : arrivals(0) {}

    void addEmail(const std::string& sender, const std::string&
subject,
                const std::string& date) {
        heap.insert(Email(sender, subject, date, arrivals++));
    }

    // NEXT: show the top email; does not remove it.
    void showNext(std::ostream& out) const {
        if (heap.isEmpty()) {
            out << "There are no emails to read." << std::endl <<
std::endl;
            return;
        }
        const Email& e = heap.peek();
        out << "Next email:" << std::endl;
        out << "\tSender: " << e.getSender() << std::endl;
        out << "\tSubject: " << e.getSubject() << std::endl;

```

```

        out << "\tDate: " << e.getDate() << std::endl << std::endl;
    }

    // READ: CEO has dealt with the top email, so remove it without
    displaying it.
    // On an empty inbox, report that there is nothing to read.
    void markRead(std::ostream& out) {
        if (heap.isEmpty()) {
            out << "There are no emails to read." << std::endl <<
std::endl;
            return;
        }
        heap.removeMax();
    }

    void showCount(std::ostream& out) const {
        out << "There are " << heap.count() << " emails to read." <<
std::endl << std::endl;
    }

private:
    MaxHeap heap;
    long arrivals;
};

//
-----
-----
// CommandProcessor: reads the test file and drives the inbox.
//
-----
-----
class CommandProcessor {
public:
    explicit CommandProcessor(CEOInbox& target) : inbox(target) {}

    void run(std::istream& in, std::ostream& out) {
        std::string line;
        while (std::getline(in, line)) {
            trim(line);
            if (line.empty()) continue;

```

```

        if (line.compare(0, 5, "EMAIL") == 0) {
            handleEmail(line.substr(5));
        } else if (line == "NEXT") {
            inbox.showNext(out);
        } else if (line == "READ") {
            inbox.markRead(out);
        } else if (line == "COUNT") {
            inbox.showCount(out);
        }
    }
}

private:
    CEOInbox& inbox;

    static void trim(std::string& s) {
        const char* ws = " \t\r\n";
        std::string::size_type b = s.find_first_not_of(ws);
        if (b == std::string::npos) { s.clear(); return; }
        std::string::size_type e = s.find_last_not_of(ws);
        s = s.substr(b, e - b + 1);
    }

    // Expects: " <category>,<subject>,<MM-DD-YYYY>"
    void handleEmail(std::string fields) {
        trim(fields);
        std::vector<std::string> parts;
        std::string piece;
        std::istringstream ss(fields);
        while (std::getline(ss, piece, ',')) {
            trim(piece);
            parts.push_back(piece);
        }
        if (parts.size() >= 3) inbox.addEmail(parts[0], parts[1],
parts[2]);
    }
};

int main(int argc, char* argv[]) {
    CEOInbox inbox;

```

```

CommandProcessor processor(inbox);

if (argc > 1) {
    std::ifstream file(argv[1]);
    if (!file) {
        std::cerr << "Could not open " << argv[1] << std::endl;
        return 1;
    }
    processor.run(file, std::cout);
} else {
    processor.run(std::cin, std::cout);
}
return 0;
}

```

Gemini:

```

#include <iostream>
#include <fstream>
#include <string>
#include <sstream>

```

// Class representing an Email entity

```

class Email {
private:
    std::string sender;
    std::string subject;
    std::string dateStr; // Format: MM-DD-YYYY

```

// Priority ordering: Boss > Subordinate > Peer > ImportantPerson > OtherPerson

```

int getSenderPriority() const {
    if (sender == "Boss") return 5;
    if (sender == "Subordinate") return 4;
    if (sender == "Peer") return 3;
    if (sender == "ImportantPerson") return 2;
    if (sender == "OtherPerson") return 1;
    return 0;
}

```

// Converts MM-DD-YYYY string to YYYYMMDD integer for chronological comparison

```

int getDateAsComparableInt() const {
    if (dateStr.length() != 10) return 0;
    int month = std::stoi(dateStr.substr(0, 2));
    int day = std::stoi(dateStr.substr(3, 2));
    int year = std::stoi(dateStr.substr(6, 4));
    return year * 10000 + month * 100 + day;
}

```

```

public:
    Email() : sender(""), subject(""), dateStr("") {}

    Email(const std::string& s, const std::string& sub, const std::string& d)
        : sender(s), subject(sub), dateStr(d) {}

    std::string getSender() const { return sender; }
    std::string getSubject() const { return subject; }
    std::string getDate() const { return dateStr; }

    bool operator<(const Email& other) const {
        int p1 = getSenderPriority();
        int p2 = other.getSenderPriority();
        if (p1 != p2) {
            return p1 < p2;
        }
        // Newer date (greater integer) takes precedence
        return getDateAsComparableInt() < other.getDateAsComparableInt();
    }

    bool operator>(const Email& other) const {
        return other < *this;
    }
};

// Custom List-Based Dynamic Array MaxHeap Implementation
class ListMaxHeap {
private:
    Email* heapArray;
    int capacity;
    int currentSize;

    void resize() {
        capacity *= 2;
        Email* newArray = new Email[capacity];
        for (int i = 0; i < currentSize; ++i) {
            newArray[i] = heapArray[i];
        }
        delete[] heapArray;
        heapArray = newArray;
    }

    void heapifyUp(int index) {
        while (index > 0) {
            int parentIndex = (index - 1) / 2;
            if (heapArray[parentIndex] < heapArray[index]) {
                std::swap(heapArray[parentIndex], heapArray[index]);
            }
        }
    }
};

```

```

        index = parentIndex;
    } else {
        break;
    }
}
}

```

```

void heapifyDown(int index) {
    while (true) {
        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;
        int largest = index;

        if (leftChild < currentSize && heapArray[largest] < heapArray[leftChild]) {
            largest = leftChild;
        }
        if (rightChild < currentSize && heapArray[largest] < heapArray[rightChild]) {
            largest = rightChild;
        }

        if (largest != index) {
            std::swap(heapArray[index], heapArray[largest]);
            index = largest;
        } else {
            break;
        }
    }
}

```

public:

```

ListMaxHeap(int initialCapacity = 10) : capacity(initialCapacity), currentSize(0) {
    heapArray = new Email[capacity];
}

```

```

~ListMaxHeap() {
    delete[] heapArray;
}

```

```

void insert(const Email& item) {
    if (currentSize == capacity) {
        resize();
    }
    heapArray[currentSize] = item;
    heapifyUp(currentSize);
    currentSize++;
}

```

```

Email getMax() const {

```

```

        if (isEmpty()) {
            return Email();
        }
        return heapArray[0];
    }

    void removeMax() {
        if (isEmpty()) return;
        heapArray[0] = heapArray[currentSize - 1];
        currentSize--;
        if (currentSize > 0) {
            heapifyDown(0);
        }
    }

    int getSize() const {
        return currentSize;
    }

    bool isEmpty() const {
        return currentSize == 0;
    }
};

```

// Priority Queue Wrapper Object

```

class PriorityQueue {
private:
    ListMaxHeap maxHeap;

public:
    void push(const Email& email) {
        maxHeap.insert(email);
    }

    void pop() {
        maxHeap.removeMax();
    }

    Email top() const {
        return maxHeap.getMax();
    }

    int size() const {
        return maxHeap.getSize();
    }

    bool empty() const {
        return maxHeap.isEmpty();
    }
}

```

```

    }
};

// Application Controller Object
class EmailProcessor {
private:
    PriorityQueue emailQueue;

public:
    void processFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error opening file: " << filename << std::endl;
            return;
        }

        std::string line;
        while (std::getline(file, line)) {
            if (line.empty()) continue;

            if (line.substr(0, 5) == "EMAIL") {
                std::string data = line.substr(6);
                std::stringstream ss(data);
                std::string sender, subject, date;

                if (std::getline(ss, sender, ',') &&
                    std::getline(ss, subject, ',') &&
                    std::getline(ss, date)) {

                    emailQueue.push(Email(sender, subject, date));
                }
            }
            else if (line == "COUNT") {
                if (emailQueue.empty()) {
                    std::cout << "There are no emails to read." << std::endl << std::endl;
                } else {
                    std::cout << "There are " << emailQueue.size() << " emails to read." << std::endl
<< std::endl;
                }
            }
            else if (line == "NEXT") {
                if (emailQueue.empty()) {
                    std::cout << "There are no emails to read." << std::endl << std::endl;
                } else {
                    Email nextEmail = emailQueue.top();
                    std::cout << "Next email:" << std::endl;
                    std::cout << "  Sender: " << nextEmail.getSender() << std::endl;
                    std::cout << "  Subject: " << nextEmail.getSubject() << std::endl;
                }
            }
        }
    }
};

```

```

        std::cout << "    Date: " << nextEmail.getDate() << std::endl << std::endl;
    }
}
else if (line == "READ") {
    if (emailQueue.empty()) {
        std::cout << "There are no emails to read." << std::endl << std::endl;
    } else {
        emailQueue.pop();
    }
}
}
}

file.close();
};

int main(int argc, char* argv[]) {
    std::string filename = "extended_input.txt";
    if (argc > 1) {
        filename = argv[1];
    }

    EmailProcessor processor;
    processor.processFile(filename);

    return 0;
}

```

Code Analysis:

Correctness: Both programs correctly implemented the array based Max-Heap and prioritize the emails in the order of priority. However, Claude's code is better because it uses a fixed arrival order to break any ties between two near important categories. It also is better as it can support both files and standard output. Hence, Claude's code is more usable.

Execution Time: Both codes have the same time complexity. Insertion and removal from MaxHeap takes $O(\log n)$, whereas accessing the highest priority email and counting emails takes $O(1)$. Thus, neither code has much advantage over the other in terms of execution time and time complexity.

Space Complexity: Both programs use $O(n)$ space because they store the emails in a dynamically allocated array. When it is resized, a space of $O(n)$ is required to copy the existing elements. Therefore, both programs have the same space complexity.

Maintainability: The second program is more maintainable because it has clearer separation of responsibilities between Email, MaxHeap, CEOInbox, and CommandProcessor. It also prevents unsafe copying of the dynamically allocated heap and avoids the unnecessary PriorityQueue wrapper used in the first program. Furthermore, only Claude's code has more

concrete comments that are helpful for running the program properly and in the future make changes.

Which Code Is Better?

The Claude code is the stronger implementation overall. Execution time and time complexity are essentially identical between the two versions, since both use an array-based MaxHeap with $O(\log n)$ insertion and removal and $O(1)$ access to the highest-priority email. Space complexity is also the same at $O(n)$. However, what the Claude code does better is in its correctness and maintainability. It ensures there are no ties between the tags of employees, more flexible output handling, and explicitly prevents unsafe copying of dynamically allocated heap. The first version is still functional and well organized, but it contains an additional PriorityQueue wrapper that adds little functionality. Overall, the Claude code is the better starting point because it provides the same performance and space efficiency while offering more robust behavior, cleaner organization, and better maintainability.

How the code was improved:

Error & Edge Case Handling

- Malformed EMAIL lines (wrong field count, unknown category, empty subject, invalid date) are skipped with a warning that names the line number, instead of entering the queue.
- Dates are checked as MM-DD-YYYY with a valid month and day range, so a bad date can never corrupt the sort order.
- NEXT and READ on an empty inbox print a single "no emails to read" message instead of failing or repeating it.
- Unknown commands, a missing input file, and out-of-memory errors are reported on the error stream so they don't mix into the normal output.

Maintainability

- Every line of code has a comment, and each class has a header comment explaining its role.
- The category order is now strict (Boss > Subordinate > Peer > ImportantPerson > OtherPerson), with the newest date first inside each category.

Improved Code:

```
// CEO Email Prioritizer
// Uses a hand-built, array-based MaxHeap (no pre-existing heap
modules).
// Build:  g++ -std=c++11 -o ceo_email ceo_email_priority.cpp
// Run:    ./ceo_email input.txt      (or pipe a file to stdin)
```

```

#include <iostream>    // std::cout, std::cerr, std::endl,
std::ostream, std::istream
#include <fstream>     // std::ifstream for reading the input file
#include <sstream>      // std::istringstream for parsing text
#include <string>       // std::string
#include <vector>       // std::vector, used only to hold split input
fields
#include <new>          // std::bad_alloc, thrown when memory runs out
#include <exception>    // std::exception, base class for caught
errors

//
-----
-----
// Email: one message plus everything needed to rank it.
//
-----
-----
class Email {
public:
// members usable from outside the class
    // Default constructor: needed so the heap can allocate an empty
array of Emails.
    Email() : rank(0), year(0), month(0), day(0), seq(0) {}           //
start every number at 0

    // Main constructor: builds an Email and works out its ranking
values.
    Email(const std::string& senderCategory, const std::string&
subjectLine,
        const std::string& dateText, long arrivalOrder)
        // Initializer list: store the text fields as given
        : sender(senderCategory), subject(subjectLine),
date(dateText),
        // rank comes from the category; the date parts are filled
in below
        rank(rankFor(senderCategory)), year(0), month(0), day(0),
seq(arrivalOrder) { //
remember arrival order for tie-breaking
        parseDate(dateText); //
split MM-DD-YYYY into month, day, year

```

```

    }
end of constructor

    // True if this email should be read BEFORE the other one.
    bool hasHigherPriorityThan(const Email& other) const {
        if (rank != other.rank) return rank > other.rank;
different categories: higher category wins
        if (year != other.year) return year > other.year;
same category: newer year wins
        if (month != other.month) return month > other.month;
same year: newer month wins
        if (day != other.day) return day > other.day;
same month: newer day wins
        return seq < other.seq;
exact tie: the one that arrived first wins
    }
end of hasHigherPriorityThan

    // Input validation helpers (used before an Email is created).
    static bool isValidCategory(const std::string& c) {
true only for the five allowed categories
        return c == "Boss" || c == "Subordinate" || c == "Peer" || //
check the first three names
            c == "ImportantPerson" || c == "OtherPerson";
then the last two
    }
end of isValidCategory

    static bool isValidDate(const std::string& text) {
checks the MM-DD-YYYY format
        int m = 0, d = 0, y = 0;
month, day, year read from the text
        char dash1 = 0, dash2 = 0, extra = 0;
the two dashes, plus a probe for leftover text
        std::istringstream in(text);
treat the string like an input stream
        if (!(in >> m >> dash1 >> d >> dash2 >> y)) return false;
fail if it is not number-dash-number-dash-number
        if (dash1 != '-' || dash2 != '-') return false;
separators must really be dashes
    }

```

```

        if (in >> extra) return false; //
fail if anything follows the year
        return m >= 1 && m <= 12 && d >= 1 && d <= 31 && y >= 1; //
month and day must be in a sensible range
    } //
end of isValidDate

    const std::string& getSender() const { return sender; } //
read-only access to the sender category
    const std::string& getSubject() const { return subject; } //
read-only access to the subject line
    const std::string& getDate() const { return date; } //
read-only access to the original date text

private:
// members hidden from outside the class
    std::string sender, subject, date; //
the text fields exactly as given
    int rank; //
numeric category priority (bigger = read sooner)
    int year, month, day; //
date split into numbers for comparing
    long seq; //
arrival order, used to break exact ties

    // Strict category order: Boss > Subordinate > Peer >
ImportantPerson >
    // OtherPerson. Within a category, the newest date wins.
    static int rankFor(const std::string& category) { //
converts a category name to a number
        if (category == "Boss") return 5; //
Boss is read first
        if (category == "Subordinate") return 4; //
then Subordinate
        if (category == "Peer") return 3; //
then Peer
        if (category == "ImportantPerson") return 2; //
then ImportantPerson
        return 1; //
OtherPerson (or unknown) is read last

```

```

    }
//
end of rankFor

    void parseDate(const std::string& text) {
//
splits MM-DD-YYYY into numbers
        int m = 0, d = 0, y = 0;
//
temporary month, day, year
        char dash1 = 0, dash2 = 0;
//
the two dashes between the numbers
        std::istringstream in(text);
//
treat the string like an input stream
        if (in >> m >> dash1 >> d >> dash2 >> y) {
//
only store the values if the read worked
            month = m; day = d; year = y;
//
save into the object's fields
        }
//
end of if
    }
//
end of parseDate
};
// end of class Email

//
-----
-----
// MaxHeap: array-based binary heap of Email objects, written from
scratch.
// Parent of i is (i-1)/2; children are 2i+1 and 2i+2.
//
-----
-----
class MaxHeap {
public:
// members usable from outside the class
    // Constructor: start with an empty heap and room for 8 emails.
    MaxHeap() : size(0), capacity(8), data(new Email[8]) {}
    ~MaxHeap() { delete[] data; }
//
destructor: free the array when the heap is destroyed

    // Copying is not needed; disable it to avoid double-deletes.

```

```

    MaxHeap(const MaxHeap&) = delete; //
forbid copy construction
    MaxHeap& operator=(const MaxHeap&) = delete; //
forbid copy assignment

    bool isEmpty() const { return size == 0; } //
true when no emails are stored
    int count() const { return size; } //
how many emails are stored

    void insert(const Email& e) { //
add a new email to the heap
        if (size == capacity) grow(); //
array is full, so make it bigger first
        data[size] = e; //
place the new email in the first free slot
        siftUp(size); //
move it up until the heap order is restored
        ++size; //
one more email is now stored
    } //
end of insert

    // Highest-priority email without removing it. Caller must check
isEmpty().
    const Email& peek() const { return data[0]; } //
the top of the heap is always slot 0

    // Removes the highest-priority email. Caller must check
isEmpty().
    void removeMax() { //
delete the top email
        --size; //
one fewer email; 'size' is now the last slot's index
        if (size > 0) { //
only reorder if emails remain
            data[0] = data[size]; //
move the last email into the top slot
            siftDown(0); //
push it down until the heap order is restored

```

```

        }
end of if
    }
end of removeMax

private:
// members hidden from outside the class
    int size;
number of emails currently stored
    int capacity;
number of slots currently allocated
    Email* data;
the dynamic array that holds the heap

    void grow() {
double the array's capacity
        int newCap = capacity * 2;
new size is twice the old size
        Email* bigger = new Email[newCap];
allocate the larger array
        for (int i = 0; i < size; ++i) bigger[i] = data[i];
copy every stored email across
        delete[] data;
free the old, smaller array
        data = bigger;
point to the new array
        capacity = newCap;
record the new capacity
    }
end of grow

    void swapAt(int a, int b) {
swap the emails in slots a and b
        Email tmp = data[a];
hold slot a's email temporarily
        data[a] = data[b];
copy slot b into slot a
        data[b] = tmp;
put the held email into slot b
    }
end of swapAt

```

```

    void siftUp(int i) { //
move the email at slot i up toward the top
        while (i > 0) { //
stop once we reach the top slot
            int parent = (i - 1) / 2; //
index of this slot's parent
            if (data[i].hasHigherPriorityThan(data[parent])) { //
child outranks its parent?
                swapAt(i, parent); //
then swap them
                i = parent; //
continue checking from the parent's slot
            } else { //
child does not outrank its parent
                break; //
heap order is fine, so stop
            } //
end of if/else
        } //
end of while
    } //
end of siftUp

    void siftDown(int i) { //
move the email at slot i down toward the bottom
        while (true) { //
loop until we break out
            int left = 2 * i + 1, right = 2 * i + 2, best = i; //
child indexes; 'best' tracks the highest priority of the three
            // If the left child exists and outranks the current
best, it becomes the best
            if (left < size &&
data[left].hasHigherPriorityThan(data[best])) best = left;
            // Same check for the right child
            if (right < size &&
data[right].hasHigherPriorityThan(data[best])) best = right;
            if (best == i) break; //
neither child outranks it, so it is in place
            swapAt(i, best); //
swap with the higher-priority child

```

```

        i = best;
continue from the child's slot
    }
end of while
}
end of siftDown
};
// end of class MaxHeap

//
-----
-----
// CEOInbox: owns the MaxHeap and implements the EMAIL / NEXT / READ
/ COUNT
// commands.
//
-----
-----
class CEOInbox {
public:
// members usable from outside the class
    CEOInbox() : arrivals(0), emptyNoticeShown(false) {}
constructor: no emails yet, empty notice not shown yet

    // EMAIL: add a new email to the inbox.
    void addEmail(const std::string& sender, const std::string&
subject,
                const std::string& date) {
        // Build an Email stamped with its arrival number, then
insert it into the heap
        heap.insert(Email(sender, subject, date, arrivals++));
        emptyNoticeShown = false;
inbox is no longer empty, so allow the notice again later
    }
end of addEmail

    // NEXT: show the top email; does not remove it.
    void showNext(std::ostream& out) {
'out' is where the text is printed
        if (heap.isEmpty()) {
nothing to show?

```

```

        printEmptyNotice(out);                                //
tell the user the inbox is empty (only the first time)
        return;                                              //
leave the function early
    }                                                        //
end of if
        const Email& e = heap.peek();                        //
look at the top email without removing it
        out << "Next email:" << std::endl;
// print the heading
        out << "\tSender: " << e.getSender() << std::endl;    //
print the sender, indented with a tab
        out << "\tSubject: " << e.getSubject() << std::endl;    //
print the subject
        out << "\tDate: " << e.getDate() << std::endl << std::endl;
// print the date, then a blank line
    }                                                        //
end of showNext

    // READ: CEO has dealt with the top email, so remove it without
displaying it.
    // On an empty inbox, report that there is nothing to read.
    void markRead(std::ostream& out) {                        //
'out' is where any message is printed
        if (heap.isEmpty()) {                                //
nothing to read?
            printEmptyNotice(out);                            //
tell the user the inbox is empty (only the first time)
            return;                                          //
leave the function early
        }                                                  //
end of if
        heap.removeMax();                                    //
delete the highest-priority email silently
    }                                                        //
end of markRead

    // COUNT: display how many unread emails remain.
    void showCount(std::ostream& out) const {                //
'out' is where the text is printed

```

```

        // Print the current number of emails, followed by a blank
line
        out << "There are " << heap.count() << " emails to read." <<
std::endl << std::endl;
    }
end of showCount

private:
// members hidden from outside the class
    MaxHeap heap;
the priority queue that stores every unread email
    long arrivals;
counter that gives each email an arrival number
    bool emptyNoticeShown;
true once the "no emails" message has been printed

    // Prints the "no emails" message only once per empty spell, so
repeated
    // NEXT/READ commands on an empty inbox do not print it again.
    void printEmptyNotice(std::ostream& out) {
'out' is where the message is printed
        if (emptyNoticeShown) return;
already told the user, so stay quiet
        out << "There are no emails to read." << std::endl <<
std::endl; // print the message and a blank line
        emptyNoticeShown = true;
remember that it has been shown
    }

end of printEmptyNotice
};
// end of class CEOInbox

//
-----
-----
// CommandProcessor: reads the test file and drives the inbox.
//
-----
-----
class CommandProcessor {

```

```

public:
// members usable from outside the class
    // Constructor: remember which inbox to send commands to.
    explicit CommandProcessor(CEOInbox& target) : inbox(target) {}

    void run(std::istream& in, std::ostream& out) { //
read commands from 'in', print results to 'out'
        std::string line; //
holds the current line of input
        int lineNo = 0; //
line counter, used in warning messages
        while (std::getline(in, line)) { //
read one line at a time until the input ends
            ++lineNo; //
count this line
            trim(line); //
strip leading and trailing whitespace
            if (line.empty()) continue; //
skip blank lines

            if (line.compare(0, 6, "EMAIL ") == 0) { //
line starts with "EMAIL " (including the space)?
                handleEmail(line.substr(6), lineNo); //
pass everything after "EMAIL " to the parser
            } else if (line == "NEXT") { //
NEXT command
                inbox.showNext(out); //
show the top email
            } else if (line == "READ") { //
READ command
                inbox.markRead(out); //
remove the top email
            } else if (line == "COUNT") { //
COUNT command
                inbox.showCount(out); //
show how many emails remain
            } else { //
anything else is not a known command
                // Warn on the error stream and keep going
                std::cerr << "Warning (line " << lineNo

```

```
<< "): unknown command ignored: " << line
<< std::endl;
    }
end of if/else chain
}
end of while
}
end of run

private:
// members hidden from outside the class
    CEOInbox& inbox;
reference to the inbox being controlled

    static void trim(std::string& s) {
removes whitespace from both ends of a string
        const char* ws = " \t\r\n";
characters counted as whitespace
        std::string::size_type b = s.find_first_not_of(ws);
position of the first non-whitespace character
        if (b == std::string::npos) { s.clear(); return; }
all whitespace: empty the string and stop
        std::string::size_type e = s.find_last_not_of(ws);
position of the last non-whitespace character
        s = s.substr(b, e - b + 1);
keep only the text between those positions
    }
end of trim

    // Expects: "<category>,<subject>,<MM-DD-YYYY>". Bad lines are
reported
    // on stderr and skipped so one bad line cannot corrupt the
inbox.

    void handleEmail(std::string fields, int lineNo) {
parse and validate one EMAIL line
        trim(fields);
remove stray whitespace around the fields
        std::vector<std::string> parts;
will hold the comma-separated pieces
        std::string piece;
the piece currently being read
```

```

        std::istringstream ss(fields);
//
// treat the string like an input stream
        while (std::getline(ss, piece, ',')) {
//
// split on commas, one piece at a time
            trim(piece);
//
// remove whitespace around this piece
            parts.push_back(piece);
//
// store it
        }
//
// end of while
        if (parts.size() != 3) {
//
// must be exactly category, subject, date
            std::cerr << "Warning (line " << lineNo
//
// report the problem with the line number
                << "): EMAIL needs 3 comma-separated fields;
//
// skipped." << std::endl;
        } else if (!Email::isValidCategory(parts[0])) {
//
// category must be one of the five allowed names
            std::cerr << "Warning (line " << lineNo
//
// report the bad category
                << "): invalid sender category '" << parts[0]
//
// << "'; skipped." << std::endl;
        } else if (parts[1].empty()) {
//
// subject must not be empty
            std::cerr << "Warning (line " << lineNo
//
// report the empty subject
                << "): empty subject line; skipped." <<
//
// std::endl;
        } else if (!Email::isValidDate(parts[2])) {
//
// date must be a valid MM-DD-YYYY
            std::cerr << "Warning (line " << lineNo
//
// report the bad date
                << "): invalid date '" << parts[2] << "' (use
//
// MM-DD-YYYY); skipped." << std::endl;
        } else {
//
// every check passed
            inbox.addEmail(parts[0], parts[1], parts[2]);
//
// add the email to the inbox
        }
//
// end of if/else chain

```

```

    }
end of handleEmail
};
// end of class CommandProcessor

int main(int argc, char* argv[]) {
// program entry; argv[1] is the optional input file name
    try {
catch any unexpected error below
        CEOInbox inbox;
create the CEO's inbox
        CommandProcessor processor(inbox);
create the processor that drives the inbox

        if (argc > 1) {
a file name was given on the command line
            std::ifstream file(argv[1]);
try to open that file
            if (!file) {
opening failed
                std::cerr << "Error: could not open " << argv[1] <<
std::endl; // tell the user which file failed
                return 1;
exit with an error code
            }
end of if
            processor.run(file, std::cout);
process the file's commands, printing to the screen
        } else {
no file name given
            processor.run(std::cin, std::cout);
read commands from standard input instead
        }
end of if/else
    } catch (const std::bad_alloc&) {
memory allocation failed
        std::cerr << "Error: out of memory." << std::endl;
report it
        return 1;
exit with an error code

```

```
        } catch (const std::exception& e) {                                //
any other standard error
        std::cerr << "Error: " << e.what() << std::endl;                //
report its message
        return 1;                                                         //
exit with an error code
    }                                                                      //
end of try/catch
    return 0;                                                             //
normal, successful exit
}
// end of main
```